RUTGERS UNIVERSITY · FALL 2026

CS 110

Conditionals, IPO-S and Flowcharts

Recitation 7



Antara Thapliyal · Recitation Instructor

Sections 1 & 2 · ARC 116 (Busch)

Today's Plan

Your next assignment needs all of this. Open your IDE now.

Plan before you code	IPO-S and flowcharts — working the problem out before typing.	**15 min**
Conditionals	if, if-else, nested if-else, and the operators that go inside them.	**25 min**
Using math and stopping early	sqrt, ceil, floor, and how to end a program when the input is bad.	**15 min**

Read the Python_03 assignment before you start it. There is a lot of arithmetic, and the planning we do today is what keeps it straight.

PART ONE

Plan Before You Code

Working the problem out before you type a single line.

About 15 minutes



IPO-S: Four Questions to Ask First

Answer these before writing code, and the code almost writes itself.

I

Input

What does the program need from the user?

FOR TODAY'S PROBLEM

`The numeric grade`

P

Process

What has to happen to turn input into output?

FOR TODAY'S PROBLEM

`Decide which letter it maps to`

O

Output

What does the program give back?

FOR TODAY'S PROBLEM

`The letter grade`

S

Storage

What variables do you need along the way?

FOR TODAY'S PROBLEM

`grade, letterGrade`

Process is the hard one. The other three are usually obvious once you read the problem.

Today's Problem, Written as IPO-S

A person inputs a numerical grade. The program outputs the matching letter grade.

```
A:   90 – 100
B:   80 – 89
C:   79 and below
```

Input

numerical grade

Output

the letter grade

Storage

`grade, letterGrade`

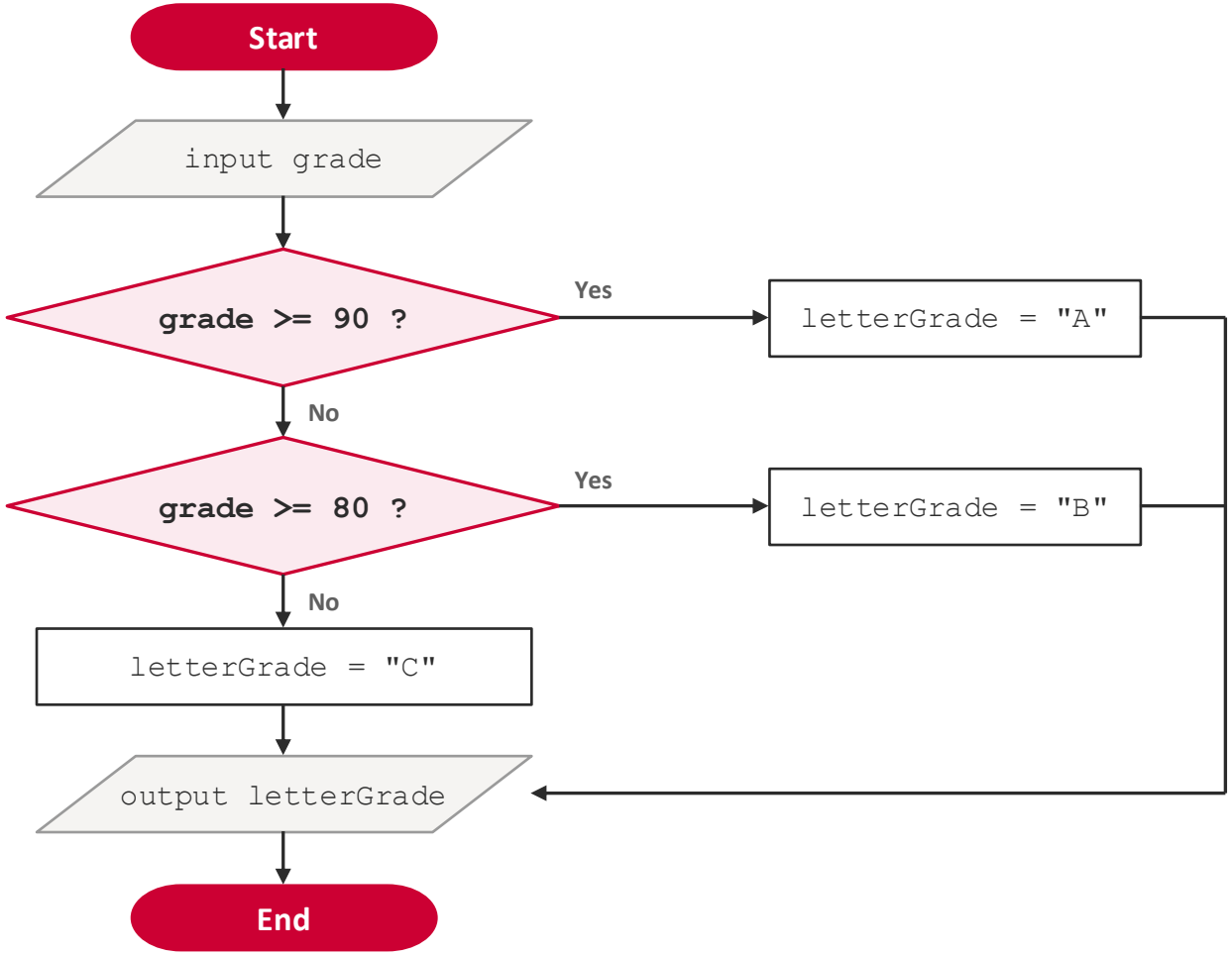
Process

```
if grade >= 90
        letterGrade = A
if grade >= 80 and grade < 90
        letterGrade = B
if grade < 80
        letterGrade = C
```

This is not Python yet — it is the plan, written so a person can read it.

The Same Plan, as a Flowchart

A picture of the decisions. Each shape means something specific.



What each shape means

Start or End

Input or output

A decision — two ways out

Something the program does

Notice there is one way in and one way out of the whole chart. Every path reaches End.

PART TWO

Conditionals

Making the program choose between different paths.

About 25 minutes



The Operators You Need

These go inside the condition — the part between if and the colon.

Relational — compare two values

`>`	greater than
`>=`	greater than or equal to
`<`	less than
`<=`	less than or equal to
`==`	equal to (two signs!)
`!=`	not equal to

Boolean — join two conditions

`and` True only when BOTH sides are true

```
grade >= 80 and grade < 90
```

`or` True when EITHER side is true

```
day == 6 or day == 0
```

Each side of and must be a complete comparison.

```
80 <= grade and grade < 90   works
80 <= grade and < 90         does not
```

Program 1: Three Separate ifs

The direct translation of the IPO-S process block.

```
# Recitation problem 1
grade = int(input("Enter the integer numeric grade: "))

if grade >= 90:
    letterGrade = "A"
if grade >= 80 and grade < 90:
    letterGrade = "B"
if grade < 80:
    letterGrade = "C"

print("Your letter grade is", letterGrade)
```

All three conditions are checked on every single run, even after the answer is known.

Sample runs

```
grade 95   ->  A
grade 85   ->  B
grade 72   ->  C
```

Watch the middle one

It needs and, because B is a range with a floor and a ceiling. Without the second half, a 95 would be relabeled B.

Program 2: Nested if-else

Same answers, different structure. Build this one line by line from Program 1.

```
# Recitation problem 2
grade = int(input("Enter the integer numeric grade: "))

if grade >= 90:
    letterGrade = "A"
else:
    if grade >= 80:
        letterGrade = "B"
    else:
        letterGrade = "C"

print("Your letter grade is", letterGrade)
```

Both programs give identical answers for every grade. Try 95, 85 and 72 in each.

else means everything else

Reaching else already tells you grade < 90. You never have to test it again.

The and disappears

Inside that else, grade >= 80 is enough to mean 80–89.

Indentation is the structure

The inner if-else is indented because it lives inside the outer else.

Which One Does Less Work?

Same answers — so compare them by counting how many comparisons each one makes.

Grade entered	Letter	Program 1	Program 2
95	**A**	4 comparisons	**1 comparison**
85	**B**	4 comparisons	**2 comparisons**
72	**C**	4 comparisons	**2 comparisons**

Where Program 1's four comparisons come from

```
grade >= 90                          →  1
grade >= 80 and grade < 90           →  2
grade < 80                           →  1
```

Four comparisons on every run, whatever the grade is.

Why this matters

Four comparisons instead of one is invisible on a single grade. Run it over a million records and the difference is real.

Two correct programs are not always equally good. That judgment is a large part of computer science.

The Trap: A Gap in Your Conditions

One wrong symbol and some inputs match nothing at all.

```
if grade > 90:          # should be >=
    letterGrade = "A"
if grade >= 80 and grade < 90:
    letterGrade = "B"
if grade < 80:
    letterGrade = "C"
print("Your letter grade is", letterGrade)
```

Enter 90 and Python stops with:

```
NameError: name 'letterGrade' is not defined
```

Program 2 cannot have this problem — an else always catches whatever the if did not.

What happened

90 is not greater than 90, so the first if fails. It is not below 90 either, so the second fails. And it is not under 80. No branch runs, letterGrade is never created, and print has nothing to show.

How to avoid it

Test the boundaries: 90, 89, 80, 79. Those four values catch almost every mistake of this kind.

PART THREE

Using math and Stopping Early

Square roots, rounding, and refusing bad input.

About 15 minutes



import math and Pythagoras

$c^2 = a^2 + b^2$, and the one line people forget.

```
import math

aStr = input("Enter value for a: ")
a = float(aStr)
bStr = input("Enter value for b: ")
b = float(bStr)

c = math.sqrt(a**2 + b**2)
print("c =", c)
```

With a = 3 and b = 4

```
c = 5.0
```

Leave out import math

and the program stops dead:

```
NameError: name 'math'
is not defined
```

Two things to notice

`**` is “to the power of”, so a**2 is a squared.

`float`, not int — lengths can have decimals.

ceil and floor, in Interactive Mode

Type one line, get one answer. The fastest way to check what a function does.

```
>>> import math
>>> math.ceil(4.2)
5
>>> math.floor(4.9)
4
```

All three rounders, side by side

Value	ceil	floor	round
4.2	5	4	4
4.5	5	4	4
4.9	5	4	5

Interactive mode

The >>> prompt runs one line at a time and prints the result immediately — no file, no Run button. Use it whenever you are unsure what something does.

ceil always goes up and floor always goes down, whatever the decimal is. round goes to the nearest — and it sends a tie like 4.5 to the even number, so round(4.5) is 4, not 5.

Stopping a Program Early

If the input makes no sense, say so and stop — do not calculate with it.

```
aStr = input("Enter a positive value for a: ")
a = float(aStr)

if a <= 0:
    print("Input error")
    quit()
```

`a = 5`	Continues to the rest of the program
`a = -2`	Input error → program ends here

Guard your input

A negative length, a zero divisor, a grade of 500 — none of these should flow into your calculation. Check them first.

Both lines must be indented under the if, or the program quits every time — even on good input.

Before You Start the Assignment

Write the IPO-S first	On paper, before any typing. Input, Process, Output, Storage.
Cast every input	input() gives text. int() or float() it before any arithmetic.
Choose if-else over stacked ifs	Fewer comparisons, and no gap for a value to fall through.
Test the boundaries	Not just a middle value — the exact numbers where the answer changes.
import math before you use it	sqrt, ceil and floor all need it.

We will walk through the Python_03 assignment together now — have it open on Canvas.

Questions?

Bring them to recitation, office hours, lab support, or Piazza.



at1679@scarletmail.rutgers.edu

Office hours: Thursdays 11:00 AM – 12:00 PM (Zoom link on Canvas)

Lab support: Wednesdays 12:00 – 2:00 PM, ARC (Busch)